Homework 4

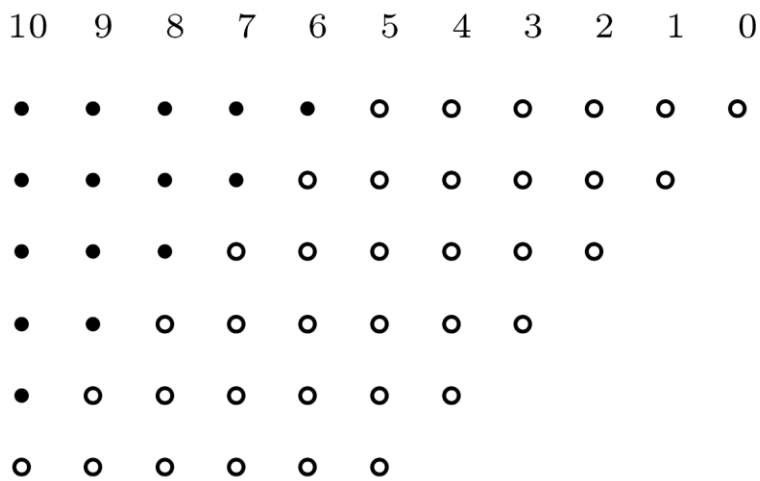
Reduction of Sign-Extension Bits

Handout: 2025/11/24

Due: 3 weeks later

1. **Reduction of Sign-Extension Bits**

Three Verilog code templates for multiplier designs are provided, unsigned, signed, and radix-4 booth-recoded to generate the partial products. *There might be bugs inside. Try to fix them if you find some.* In general, the multiplier design consists of partial product bit (ppb) generation (including sign extension, and/or Booth recoding) and partial product accumulation. For example, the following figure shows a ppb matrix with n=6 rows of partial prouducts (including the sign-extenstion bits highlighted by black dots). The partial product reduction can be efficiently implemented using carry save addition (CSA) to reduce the ppb height to two.



There are two ways of realizing CSA, row-based and column based. The row-based CSA concurrently takes three rows of partial products and reduces to two rows of sum and carry using (3:2) counters or (2:2) counters. That is, the reduction process is in unit of row of partial products which are then passed to the next CSA level. The column – based CSA takes three (or two) and reduces to 2 output bits at the current column and next higher column using a (3:2) counter (or (2:2) counter) starting from the least significant column. The input bits of smaller delay have higher priority to be compressed in order to reduce the critical path delay. However, for ease of coding, the provided Verilog codes perform partial product reduction using for-lop to sum up all the product rows.These Verilog codes of multiplier designs can be viewed as fast multipliers if the partial product accumulation using carry save addition (CSA)

followed by proper carry propagate adder (CPA) which could be achieved by proper design constraints in Synopsys Design Compiler.

Modify the partial product generation part of the codes of the three multiplier designs to simplify the sign extension bits at the upper left part of the partial product bit array. Refer to the course slides below for the reduction of sign-extension bits using three different methods:

(1) regular pp bit array

(2) modified Baugh-Wooley

(3) radix-4 Booth recoding

Compare the area/delay/power of the simplified versions with those of the original designs of non-sign-extension-bit-reduction and with those directly using the Verilog multiplier operator * for signed data or unsigned data Verilog code segments for the unsigned and signed Verilog multiplier operators are shown below. You need to compare at least the following different precisions: 8x8, 16x8,16x16.

You can consider other precisions of 4x4 and 4x8 to get extra bonus score. That is the reason why it is better to make your Berilog code flexible by parameterizing the codes.

```
// Verilog unsigned multiplier
wire [m-1:0] a;
wire [n-1:0] b;
wire [m+n-1:0] p;
assign p = a*b;
```

```
// Verilog signed multiplier
wire signed [m-1:0] a;
wire signed [n-1:0] b;
wire signed [m+n-1:0] p;
assign p = a*b;
```

Fill in the following summarized tables for three different synthesis constraints: area-optimization, delay-optimization and in-between (mid). In the first row of the following table for Verilog signed multiplier operator, also include unsigned results of unsigned operator in the parentheses for clear comparison of synthesis results for signed multiplier and unsigned multiplier.

(1) reduction of sign-extension bits: method-1

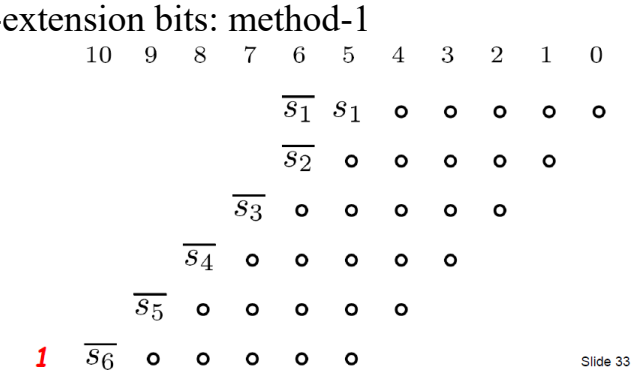


Fig. 1. Reduction of sign-bit-extension-bit partial-product-bit (ppb) array.

(2) reduction of sign-extension bits: method-2 (modified Baugh-Wooley)

Using the Negative-Weight Property of the Sign Bit

Sign extension is a way of converting negatively weighted bits (negabits) to positively weighted bits (posibits) to facilitate reduction, but there are other methods of accomplishing the same without introducing a lot of extra bits

Baugh and Wooley have contributed two such methods

Baugh-Wooley
2's-complement multiplication.

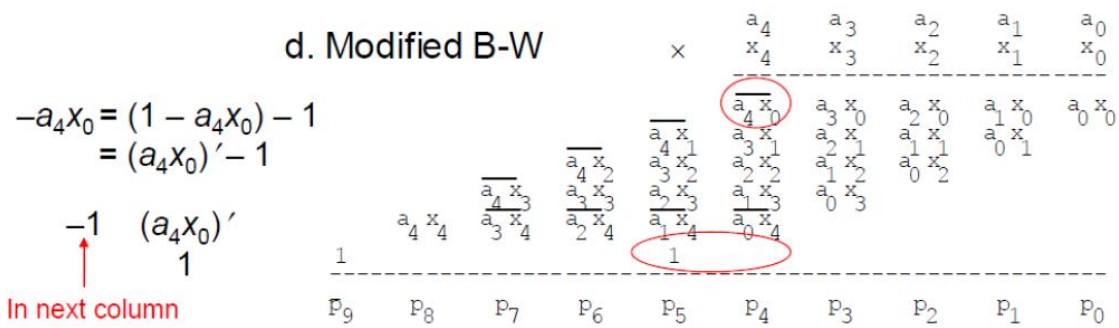
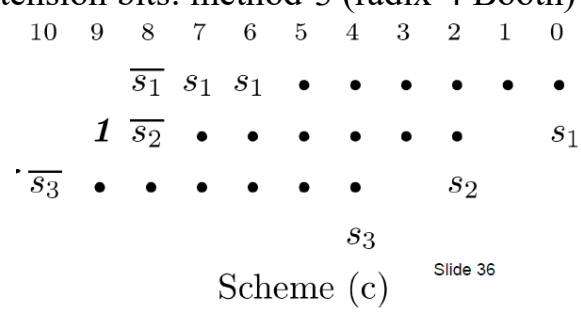


Fig. 2. Modified Baugh-Wooley.

(3) reduction of sign-extension bits: method-3 (radix-4 Booth)



Scheme (c)

Fig. 3. Radix-4 Booth.

mxn signed		area(um2)	time(ns)	power(uW)		
		CL(total)	delay	dynamic	leakage	total
operator (direct Verilog *)	area					
	mid					
	delay					
row (pp row accumulation)	area					
	mid					
	delay					
row_opt (optimized sign_ext.)	area					
	mid					
	delay					
row_mbw (modified BaughWooley)	area					
	mid					
	delay					
row_booth (radix-4 Booth)	area					
	mid					
	delay					
row_booth_opt (optimized sign ext.)	area					
	mid					
	delay					